

Revision Notes for CBSE Class 12 Computer Science Chapter 2

File Handling in Python

In Python programming, files are used to store data permanently on a secondary storage device. Unlike variables that hold temporary data during program execution, files let you save input and output for future use, reducing the need for repetitive data entry. Organizations commonly use files to keep records of employees, sales, and inventory.

A file is simply a named location on storage where data is kept for later retrieval. Python files typically have the extension .py, but data can also be stored in files with extensions like .txt, .csv, or as binary files. Storing data in files allows easy access and reusability.

Types of Files in Python

Python uses two main types of files: text files and binary files.

- **Text files** contain data in human-readable form using characters, numbers, and symbols. Examples include .txt, .py, and .csv.
- **Binary files** hold data as raw bytes and are not human-readable. They store formats like images, audio, videos, or serialized program data.

Each line in a text file is separated by an end-of-line character such as `\n`. Binary files store data in a way specific to the application, so editing them with a text editor usually shows random or unreadable symbols.

Opening and Closing Files

To work with files in Python, you first need to open them using the `open()` function. This function returns a file object, which you then use to read from or write to the file.

- The syntax is: `file_object = open(file_name, access_mode)`

- Common file modes: **r** (read), **w** (write, overwrites file), **a** (append), **rb** and **wb** (binary read/write), **r+** (read and write), **w+** (write and read), and **a+** (append and read).
- Closing a file with `close()` is important as it frees up resources and ensures data is saved correctly.

Alternatively, you can use the `with` statement for opening files. This automatically closes the file when the block is exited, reducing the risk of data loss.

Writing Data to a Text File

To write information to a file, you must open it in write (`w`) or append (`a`) mode. Python provides two primary methods:

- **write()**: Adds a single string to the file. Numbers must be converted to strings before writing.
- **writelines()**: Writes a list or tuple of strings to the file. To separate lines, include `\n` at the end of each string.

For both methods, text is initially written to a buffer. The actual write to permanent storage happens when you close the file or use `flush()`.

Reading Data from a Text File

Reading can be performed with file modes like `r`, `r+`, `w+`, or `a+`. There are three ways to read file contents:

- **read([n])**: Reads `n` bytes or the entire file if `n` is not specified.
- **readline([n])**: Reads one line from the file, or up to `n` bytes.
- **readlines()**: Reads all lines and returns them as a list, each ending with a newline character.

You can also loop over a file object directly to read line by line, which is helpful for processing large files.

To split each line into words, use `split()` on a line; to split lines into a list without newline characters, use `splitlines()`.

File Pointer Position: `seek()` and `tell()`

Python allows random access in files using `seek()` and `tell()`.

- `tell()` shows the current position of the file pointer in bytes from the start.
- `seek(offset, reference_point)` moves the file pointer to a new location. The *reference_point* can be the file's start (0), current position (1), or end (2).

These functions are useful for navigating large files when you do not want to read them sequentially.

Creating and Traversing Text Files

You can create and add data to a text file by opening it in `w+` (write and read) mode. To traverse and display a file's content, open it in `r` mode and read each line in a loop.

Performing both reading and writing in the same file is possible when you use `w+` mode. After writing data, reposition the file pointer to the start using `seek(0)` before reading.

Pickle Module: Storing Python Objects

Sometimes, you need to store complex objects such as lists and dictionaries. The **pickle** module serializes and deserializes Python objects into binary files.

- **Pickling (serialization)**: Converts a Python object to a byte stream using `pickle.dump(object, file)`. The file is opened in `wb` mode (write binary).

- **Unpickling (deserialization):** Restores a Python object from a byte stream using `pickle.load(file)`, with the file in rb mode (read binary).

Pickle is especially useful for storing structured data such as employee records or maintaining state between program executions. You can store and retrieve lists, tuples, and dictionaries efficiently.

Key Points to Remember

- A file stores data permanently, making it accessible beyond program execution.
- Text files are human-readable; binary files are not.
- Close files after use to ensure all data is written and to release resources.
- Always convert numbers to strings before writing to text files.
- Use `seek()` and `tell()` for random access within files.
- Pickling makes storing and retrieving complex objects possible in binary format.

Practicing file handling in Python is important for CBSE Class 12 Computer Science students because it prepares you for real-world programming and helps secure marks in board and practical exams. Understanding file modes, basic reading and writing, and pickling lays a strong foundation for further study in computer applications.